FAS Silicon Academy

Assertions

interview questions

SYSTEM VERILOG

1. What is an assertion in SystemVerilog?

An assertion is a verification construct used in SystemVerilog to validate the behavior of a digital design. It checks whether a specific condition or property holds true during simulation. Assertions are primarily used to detect and localize bugs early by automatically flagging design violations, helping to enforce protocol compliance and design intent. They enhance debugging efficiency and help ensure design correctness.

2. What are the types of assertions?

SystemVerilog supports two primary types of assertions:

- **Immediate Assertions**:
  These are evaluated at the point of execution and are used inside procedural blocks like `initial`, `always`, or `always_ff`. They check conditions that should hold true immediately.
- **Concurrent Assertions**:
  These are evaluated over multiple clock cycles and are written using properties and sequences. They monitor the behavior of signals over time, making them ideal for protocol checking.

3. What is the syntax of an immediate assertion?

```
assert (expression) else $error("Condition failed");
```

This statement checks the boolean expression at that moment in simulation. If the condition fails, an error message is printed. Useful for quick checks inside testbenches or design blocks.

4. Difference between `assert`, `assume`, and `cover`?

Keyword	Purpose
`assert`	Validates the design behavior (simulation & formal)
`assume`	Used mainly in formal verification to constrain the environment
`cover`	Monitors if a condition or behavior occurs for coverage analysis

5. Where are immediate assertions used?

Immediate assertions are placed within procedural blocks (`initial`, `always`, `task`, etc.). They are used to perform immediate checks at a specific simulation time. For example, checking if a signal is high when another event occurs. They are useful for quick and early bug detection.

6. What is the syntax of a concurrent assertion?

```
property p1;
  @(posedge clk) req |=> ack;
endproperty
```

assert property (p1);

Here, the property named `p1` is triggered on a rising clock edge. It checks that if `req` is true in one cycle, `ack` must be true in the same or a future cycle (depending on implication type). Concurrent assertions are ideal for protocol verification.

7. What is the difference between |−> and |=>?

|−> (Non-overlapped implication):
Checks that the consequence happens strictly in the **next cycle or later** after the antecedent.

|=> (Overlapped implication):
Checks that the consequence can occur in the **same cycle** as the antecedent.

8. What is a sequence in SVA?

A **sequence** is a construct that describes a **series of signal events or conditions over time**. Sequences are used to specify temporal behavior, such as "signal A must be followed by signal B within 3 cycles." Sequences can be reused and composed to build complex protocol behaviors.

9. What is a property in SVA?

A **property** defines a rule or behavior that a system must follow. It is often built using one or more sequences. A property can then be bound to an assertion (`assert property`) or assumption (`assume property`) to validate or constrain behavior during verification.

10. Explain disabling assertions?

disable iff (reset);

The `disable iff` clause temporarily disables the assertion when a condition is true (commonly during reset). This prevents false assertion failures when the design is in an unstable or undefined state. It is commonly used in synchronous protocols to ignore checks during reset conditions.

11. Write a concurrent assertion to check that `ack` follows `req` after exactly 2 cycles?

assert property (@(posedge clk) req |-> ##2 ack);

This assertion states that if `req` is true in a cycle, then `ack` must become true **exactly** two cycles later. This type of check is useful for timing-sensitive protocols where response latency is fixed.

12. How to use `first_match` in assertions?

(seq1 ##[1:3] seq2)[->1]

This pattern means that after `seq1`, `seq2` must occur between 1 to 3 cycles, but only the **first occurrence** is considered. It avoids repeated matches and focuses only on the earliest satisfaction of the condition.

13. What is `within` in SystemVerilog assertions?

(seq2 within seq1)

The `within` operator checks if `seq2` occurs entirely **within the time span of `seq1`**. It is used to bound behavior checks within a specific window of activity or phase in a protocol.

14. How are assertions synthesized?

Most assertions (especially concurrent assertions) are **not synthesizable** and are used only in simulation or formal verification environments. However, some **immediate assertions** may be synthesizable depending on the tool and usage. Assertions are primarily meant for **verification**, not for hardware implementation.

15. How do assertions help in debugging?

Assertions automatically monitor the design behavior and flag errors **at the moment a violation occurs**. This reduces the time and effort required to trace bugs manually. They pinpoint the exact cycle and condition that failed, making them essential for protocol checking, corner-case coverage, and reducing overall debug effort.

16. Write an assertion to check that `PSEL` is high for at least 1 cycle before `PENABLE` is asserted?

```
property psel_before_penable;
  @(posedge PCLK)
    PSEL && !PENABLE |=> ##1 PENABLE;
endproperty
assert property (psel_before_penable);
```

17. Write an assertion to ensure `PWRITE` remains stable once `PENABLE` is asserted?

```
property pwdata_stable_during_penable;
  @(posedge PCLK)
    PSEL && PENABLE && PWRITE |-> $stable(PWDATA);
endproperty
assert property (pwdata_stable_during_penable);
```

18. Write an assertion to ensure `PWDATA` remains stable during the `PENABLE` phase of a write transaction?

```
property pwdata_stable_during_penable;
  @(posedge PCLK)
    PSEL && PENABLE && PWRITE |-> $stable(PWDATA);
endproperty
assert property (pwdata_stable_during_penable);
```

19. Check that PREADY is not asserted before PENABLE?

```
property pready_after_penable;
  @(posedge PCLK)
    PREADY |-> PENABLE;
endproperty
assert property (pready_after_penable);
```

20. Assert that PADDR remains stable throughout an APB transaction?

```
property paddr_stable_transaction;
  @(posedge PCLK)
    PSEL ##1 PENABLE |-> $stable(PADDR);
endproperty
assert property (paddr_stable_transaction);
```

21. Ensure that a transfer (PSEL high) is followed by PENABLE in the next cycle?

```
property enable_after_psel;
  @(posedge PCLK)
    PSEL && !PENABLE |-> ##1 PENABLE;
endproperty
assert property (enable_after_psel);
```

22. Assert that read data (PRDATA) is valid only when PREADY is high and PWRITE is low?

```
property read_data_valid;
  @(posedge PCLK)
    PSEL && PENABLE && !PWRITE && PREADY |-> !$isunknown(PRDATA);
endproperty
assert property (read_data_valid);
```

23. Assert that AWVALID and AWREADY handshake completes in one cycle?

```
property aw_handshake;
  @(posedge ACLK)
    AWVALID ##1 AWREADY |-> AWVALID && AWREADY;
endproperty
assert property (aw_handshake);
```

24. Ensure that AWADDR remains stable after AWVALID until AWREADY is high?

```
property awaddr_stable_during_handshake;
  @(posedge ACLK)
    AWVALID && !AWREADY |=> $stable(AWADDR);
endproperty
assert property (awaddr_stable_during_handshake);
```

25. Write assertion for WVALID/WREADY handshake?

```
property wvalid_ready_handshake;
  @(posedge ACLK)
    WVALID |-> eventually WREADY;
endproperty
assert property (wvalid_ready_handshake);
```

26. Ensure WDATA is stable while WVALID is high and WREADY is low?

```
property wdata_stable_when_waiting;
  @(posedge ACLK)
    WVALID && !WREADY |=> $stable(WDATA);
endproperty
assert property (wdata_stable_when_waiting);
```

27. Check that BRESP is only valid when BVALID is high?

```
property bresp_validity;
  @(posedge ACLK)
    BVALID |-> !$isunknown(BRESP);
endproperty
assert property (bresp_validity);
```